Bacharelado em Ciência da Computação

GBC043 Sistemas de Banco de Dados

Profa. Maria Camila Nardini Barioni

Leonardo Henrique Carvalho Oliveira – 12311BCC003

Victor Gava Senema – 12211BCC041

Willian Henrique Vilela Casa Grandi – 12211BCC001

1) Utilizando o PgAdmin execute os comandos abaixo para a criação do BD que será utilizado na atividade:

- crie o esquema vendas utilizando o script fornecido com as tabelas CLIENTE, VENDEDOR, PRODUTO, PEDIDO e ITEMPED e a carga inicial de dados;
- execute comandos SELECT para ver o conteúdo de cada tabela

```
SELECT * FROM CLIENTE;
```

	codcli [PK] integer	nomecli character varying (50)	endereco character varying (60)	cidade character (20)	cep character (10)	uf character (2)	cgc character (14)
1	20	BETH DA SILVA	AV. CLIMERIO, 230	SAO PAULO	25678300	SP	32485211125025
2	110	JORGE BRITO	RUA CAIAPO, 13	CURITIBA	30078500	PR	14512764983491
3	130	EDMAR CAMARGO	RUA DA PRAIA, S/N	SALVADOR	30099300	BA	23456890000287
4	157	PAULO SOUSA	TV. MORAES, 230	LONDRINA	89785200	PR	12748111000125
5	180	LVIO FIGUEIRAS	AV.BEIRA MAR, 1256	FLORIANPOLIS...	30001500	SC	12788888001020
6	222	LUCIA CAMATA	R.ITABIRA, 123 - LOJA 9	BELO HORIZO...	22124391	MG	28315213934881
7	234	JOS OLIVEIRA	QUADRA 3, BL.3 - SL.2	BRASLIA	22897100	DF	21768811000120
8	260	SUSANA OLIVEIRA	R.LOPES MENDES, 200	NITERI	30045500	RJ	21758855000210

```
SELECT * FROM VENDEDOR;
```

	codvend [PK] integer	nomevend character varying (50)	salfixo double precision	faixacomis character (1)
1	11	JOAO SANTOS OLIVEIRA	2780	C
2	101	JOAO SANTOS SILVA	2650	C
3	111	CARLOS VIEIRA	2490	A
4	209	JOSE FELISBERTO	1800	C
5	213	JONAS SILVA	2300	A
6	240	ANTONIO MORAIS	9500	C
7	250	MAURICIO SERRA	2930	B
8	310	JOSIAS DA SILVA	870	B

```
SELECT * FROM PRODUTO;
```

	codprod [PK] integer	unidade character (2)	descri character varying (30)	valunit double precision
1	13	G	OURO	6.18
2	22	M	LINHO	0.11
3	25	KG	QUEIJO	0.97
4	30	AS	AUCAR	0.3
5	31	BA	CHOCOLATE	0.87
6	45	M	MADEIRA	0.25
7	53	M	LINHA	1.8
8	77	M	PAPEL	1.05

```
SELECT * FROM PEDIDO;
```

	numped [PK] integer	prazoentr integer	codcli integer	codvend integer	qtd_itens integer
1	300	20	180	720	0
2	301	60	260	209	0
3	302	30	110	720	0
4	303	20	20	101	0
5	304	45	830	720	0
6	305	30	871	310	0
7	306	10	410	213	0
8	307	20	390	720	0

```
SELECT * FROM ITEMPED;
```

	numped [PK] integer	codprod [PK] integer	quant integer
1	300	22	229
2	300	25	295
3	300	31	146
4	300	45	223
5	300	53	109
6	300	77	168
7	301	53	208
8	301	77	49

2) Escreva gatilhos para:

a) Realização de auditoria nos salários dos vendedores.

- Para isso, deve-se criar uma nova tabela chamada auditoria_salarios com as seguintes informações: código do vendedor, seu salário anterior, novo salário, data alteração.

- Crie o gatilho (i.e. trigger) que, ao alterar o salário de um vendedor preencha automaticamente os campos da tabela de auditoria.

- Para testar o gatilho faça um procedimento armazenado que aumenta os salários de todos os empregados de acordo com a faixa de comissão que eles recebem: 5% faixa A, 10% faixa B; 15% para as demais faixas.

- Mostrar a tabela auditoria (após os aumentos).

TABELA VENDEDOR ANTES DA ATUALIZAÇÃO DE SALÁRIOS:

	codvend [PK] integer	nomevend character varying (50)	salfixo double precision	faixacomis character (1)
1	11	JOAO SANTOS OLIVEIRA	2780	C
2	101	JOAO SANTOS SILVA	2650	C
3	111	CARLOS VIEIRA	2490	A
4	209	JOSE FELISBERTO	1800	C
5	213	JONAS SILVA	2300	A
6	240	ANTONIO MORAIS	9500	C
7	250	MAURICIO SERRA	2930	B
8	310	JOSIAS DA SILVA	870	B
9	720	FELIPE NASCIMENTO	4600	A
10	100	MARCOS FARIAS	800	D
11	200	MONICA DA SILVA	1600	D
12	280	SAMANTA DE OLIVEIRA	3600	C

CRIAÇÃO DA TABELA auditoria_salarios, GATILHO E PROCEDIMENTO ARMAZENADO:

```
CREATE TABLE auditoria_salarios (
    id SERIAL PRIMARY KEY,
    cod_vend INT NOT NULL,
    salario_anterior DECIMAL(10,2) NOT NULL,
    novo_salario DECIMAL(10,2) NOT NULL,
    data_alteracao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE OR REPLACE FUNCTION auditoria_dos_salarios()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO auditoria_salarios (cod_vend, salario_anterior, novo_salario, data_alteracao)
    VALUES (OLD.codvend, OLD.salfixo, NEW.salfixo, NOW());
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER auditar_salarios
AFTER UPDATE ON VENDEDOR
FOR EACH ROW
WHEN (OLD.salfixo IS DISTINCT FROM NEW.salfixo)
EXECUTE FUNCTION auditoria_dos_salarios();
```

```sql
CREATE OR REPLACE FUNCTION atualizar_salarios()
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    vendedor Vendedor%ROWTYPE;
BEGIN
    FOR vendedor IN SELECT * FROM Vendedor LOOP
        IF vendedor.FaixaComis = 'A' THEN
            UPDATE Vendedor
            SET Salfixo = vendedor.Salfixo * 1.05
            WHERE CodVend = vendedor.CodVend;
        ELSIF vendedor.FaixaComis = 'B' THEN
            UPDATE Vendedor
            SET Salfixo = vendedor.Salfixo * 1.10
            WHERE CodVend = vendedor.CodVend;
        ELSE
            UPDATE Vendedor
            SET Salfixo = vendedor.Salfixo * 1.15
            WHERE CodVend = vendedor.CodVend;
        END IF;
    END LOOP;
END;
$$;
```

TABELA auditoria_salarios ANTES DA EXECUÇÃO(VAZIA):

	id [PK] integer	cod_vend integer	salario_anterior numeric (10,2)	novo_salario numeric (10,2)	data_alteracao timestamp without time zone

APÓS A EXECUÇÃO:

	id [PK] integer	cod_vend integer	salario_anterior numeric (10,2)	novo_salario numeric (10,2)	data_alteracao timestamp without time zone
1	1	11	2780.00	3197.00	2025-04-04 17:52:36.827653
2	2	101	2650.00	3047.50	2025-04-04 17:52:36.827653
3	3	111	2490.00	2614.50	2025-04-04 17:52:36.827653
4	4	209	1800.00	2070.00	2025-04-04 17:52:36.827653
5	5	213	2300.00	2415.00	2025-04-04 17:52:36.827653
6	6	240	9500.00	10925.00	2025-04-04 17:52:36.827653
7	7	250	2930.00	3223.00	2025-04-04 17:52:36.827653
8	8	310	870.00	957.00	2025-04-04 17:52:36.827653
9	9	720	4600.00	4830.00	2025-04-04 17:52:36.827653
10	10	100	800.00	920.00	2025-04-04 17:52:36.827653
11	11	200	1600.00	1840.00	2025-04-04 17:52:36.827653
12	12	280	3600.00	4140.00	2025-04-04 17:52:36.827653

(b) Estava previsto no MER ter um campo com a quantidade de itens de pedido na tabela pedido (atributo derivado).

(a) Incluir esse novo atributo na tabela pedido.

```
ALTER TABLE PEDIDO
ADD COLUMN QTD_ITENS INTEGER DEFAULT 0;
```

(b) faça um procedimento armazenado para atualizar o valor deste atributo.

```
CREATE OR REPLACE PROCEDURE atualiza_qtd_itens()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE PEDIDO p
    SET QTD_ITENS = (
        SELECT COUNT(*)
        FROM ITEMPED i
        WHERE i.NUMPED = p.NUMPED
    );
END;
$$;
```

(c) crie um trigger que cuide da integridade deste campo – ou seja, que garanta que o valor ali presente corresponda sempre ao número real de itens de pedidos registrados (não pode chamar o procedimento criado em (b), pois seria muito ineficiente).

```
CREATE OR REPLACE FUNCTION atualiza_qtd_itens_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE PEDIDO
        SET QTD_ITENS = QTD_ITENS + 1
        WHERE NUMPED = NEW.NUMPED;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE PEDIDO
        SET QTD_ITENS = QTD_ITENS - 1
        WHERE NUMPED = OLD.NUMPED;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_atualiza_qtd_itens
AFTER INSERT OR DELETE ON ITEMPED
FOR EACH ROW
EXECUTE FUNCTION atualiza_qtd_itens_trigger();
```

(d) Realize inserções na base para observar o disparo do trigger (mostrar os resultados).

```
INSERT INTO CLIENTE VALUES
(2, 'Ana Costa', 'Av Central, 500', 'Rio de Janeiro', '20000-000', 'RJ', '98765432109876'),
(3, 'Pedro Santos', 'Rua B, 200', 'Belo Horizonte', '30123-456', 'MG', '45678912345678');


INSERT INTO VENDEDOR VALUES
(2, 'Carlos Lima', 2500.00, 'B'),
(3, 'Fernanda Oliveira', 1800.00, 'C');


INSERT INTO PRODUTO VALUES
(3, 'KG', 'Produto C', 15.75),
(4, 'LT', 'Produto D', 8.90),
(5, 'CX', 'Produto E', 25.00);

INSERT INTO PEDIDO (NUMPED, PRAZOENTR, CODCLI, CODVEND, QTD_ITENS) VALUES
(2, 15, 2, 2, 0),
(3, 7, 3, 3, 0);
```

```
SELECT * FROM PEDIDO WHERE NUMPED IN (2, 3)
```

	numped [PK] integer	prazoentr integer	codcli integer	codvend integer	qtd_itens integer
1	2	15	2	2	0
2	3	7	3	3	0

```
INSERT INTO ITEMPED VALUES
(2, 3, 10),
(2, 4, 8),
(3, 5, 2);

SELECT * FROM PEDIDO WHERE NUMPED IN (2, 3);
```

	numped [PK] integer	prazoentr integer	codcli integer	codvend integer	qtd_itens integer
1	2	15	2	2	2
2	3	7	3	3	1

```
INSERT INTO ITEMPED VALUES (2, 5, 4);

SELECT * FROM PEDIDO WHERE NUMPED IN (2, 3);
```

	numped [PK] integer	prazoentr integer	codcli integer	codvend integer	qtd_itens integer
1	2	15	2	2	3
2	3	7	3	3	1

```
DELETE FROM ITEMPED WHERE NUMPED = 2 AND CODPROD = 4;

SELECT * FROM PEDIDO WHERE NUMPED IN (2, 3);
```

	numped [PK] integer	prazoentr integer	codcli integer	codvend integer	qtd_itens integer
1	2	15	2	2	2
2	3	7	3	3	1

(c) Implementar a restrição de que um pedido não pode conter mais do que 10 itens. Tente violar essa restrição e mostre os resultados.

```
CREATE OR REPLACE FUNCTION atualiza_qtd_itens_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF (SELECT QTD_ITENS FROM PEDIDO WHERE NUMPED = NEW.NUMPED) >= 10 THEN
            RAISE EXCEPTION 'ja existe 10 itens!!!!!.', NEW.NUMPED;
        END IF;
        UPDATE PEDIDO
        SET QTD_ITENS = QTD_ITENS + 1
        WHERE NUMPED = NEW.NUMPED;
    ELSIF TG_OP = 'DELETE' THEN
        UPDATE PEDIDO
        SET QTD_ITENS = QTD_ITENS - 1
        WHERE NUMPED = OLD.NUMPED;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_atualiza_qtd_itens
AFTER INSERT OR DELETE ON ITEMPED
FOR EACH ROW
EXECUTE FUNCTION atualiza_qtd_itens_trigger();
```

```
SELECT NUMPED, QTD_ITENS FROM PEDIDO WHERE NUMPED = 2000;
```

	numped [PK] integer	qtd_itens integer
1	2000	0

```sql
SELECT NUMPED, QTD_ITENS FROM PEDIDO WHERE NUMPED = 2000;
```

	numped [PK] integer	codprod [PK] integer	quant integer

```sql
DO $$
BEGIN
    FOR i IN 1..10 LOOP
        INSERT INTO ITEMPED (NUMPED, CODPROD, QUANT)
        VALUES (2000, i, 1);
    END LOOP;
END $$;
```

```sql
SELECT NUMPED, QTD_ITENS FROM PEDIDO WHERE NUMPED = 2000;
```

	numped [PK] integer	qtd_itens integer
1	2000	0

```sql
SELECT NUMPED, CODPROD, QUANT FROM ITEMPED WHERE NUMPED = 2000 ORDER BY CODPROD;
```

	numped [PK] integer	codprod [PK] integer	quant integer
1	2000	1	1
2	2000	2	1
3	2000	3	1
4	2000	4	1
5	2000	5	1
6	2000	6	1
7	2000	7	1
8	2000	8	1
9	2000	9	1
10	2000	10	1

```sql
INSERT INTO ITEMPED (NUMPED, CODPROD, QUANT)
VALUES (2000, 11, 2);
```

```sql
SELECT NUMPED, QTD_ITENS FROM PEDIDO WHERE NUMPED = 2000;
```

	numped [PK] integer	qtd_itens integer
1	2000	0

```sql
SELECT NUMPED, CODPROD, QUANT FROM ITEMPED WHERE NUMPED = 2000 ORDER BY CODPROD;
```

	numped [PK] integer	codprod [PK] integer	quant integer
1	2000	1	1
2	2000	2	1
3	2000	3	1
4	2000	4	1
5	2000	5	1
6	2000	6	1
7	2000	7	1
8	2000	8	1
9	2000	9	1
10	2000	10	1
11	2000	11	2